

SOLID Principles Implementation Summary

Overview

This document summarizes the refactoring work done to fix SOLID principle violations in the Chicken Processing backend system. The original `UserService` class violated multiple SOLID principles by having too many responsibilities and dependencies on concrete implementations.

Identified Violations

1. Single Responsibility Principle (SRP) Violations

- **Original Issue:** `UserService` handled CRUD operations, authentication, password management, account management, and logging
- **Impact:** Changes to one responsibility could affect others; difficult to test and maintain

2. Interface Segregation Principle (ISP) Violations

- **Original Issue:** `UserServiceInterface` was a "fat interface" with methods for all responsibilities
- **Impact:** Clients depending on the interface had to implement methods they didn't use

3. Dependency Inversion Principle (DIP) Violations

- **Original Issue:** Direct dependencies on concrete `firestore` and `bcrypt` implementations
- **Impact:** Difficult to swap implementations; tight coupling to external libraries

4. Open/Closed Principle (OCP) Violations

- **Original Issue:** Adding new user types or authentication methods required modifying existing code
- **Impact:** High risk of breaking existing functionality when extending

Solution Architecture

New Service Structure

```
services/  
├── interfaces/  
│   ├── IUserReadService.ts      # Read operations interface  
│   ├── IUserWriteService.ts     # Write operations interface  
│   ├── IAuthenticationService.ts # Authentication interface  
│   ├── IAccountManagementService.ts # Account management interface  
│   └── IPasswordService.ts      # Password operations interface  
├── implementations/  
│   ├── UserReadService.ts       # Read operations implementation  
│   ├── UserWriteService.ts      # Write operations implementation  
│   ├── AuthenticationService.ts # Authentication implementation  
│   ├── AccountManagementService.ts # Account management implementation  
│   └── PasswordService.ts       # Password operations implementation
```

```

└─ UserService.ts                # Facade delegating to specialized services

repositories/
└─ IUserRepository.ts           # Repository interface (abstraction)
└─ FirestoreUserRepository.ts   # Firestore implementation (concrete)

config/
└─ container.ts                 # Dependency injection container

```

Implementation Details

1. Interface Segregation (ISP)

Created focused interfaces instead of one fat interface:

IUserReadService.ts

```

export interface IUserReadService {
  getById(id: string): Promise<User | null>;
  getByEmail(email: string): Promise<User | null>;
  getByFirebaseUid(firebaseUid: string): Promise<User | null>;
  list(filter?: UserFilter): Promise<User[]>;
  exists(email: string): Promise<boolean>;
}

```

IUserWriteService.ts

```

export interface IUserWriteService {
  create(email: string, userType: UserType, profileData: any): Promise<User>;
  update(id: string, updates: UpdateUserRequest): Promise<User>;
  delete(id: string): Promise<void>;
}

```

IAuthenticationService.ts

```

export interface IAuthenticationService {
  verifyCredentials(email: string, password: string): Promise<User>;
  recordLogin(userId: string, ipAddress?: string): Promise<void>;
  recordFailedLogin(userId: string): Promise<void>;
  resetLoginAttempts(userId: string): Promise<void>;
  isAccountLocked(userId: string): Promise<boolean>;
}

```

IAccountManagementService.ts

```
export interface IAccountManagementService {
  approve(userId: string, approvedBy: string): Promise<void>;
  reject(userId: string, reason: string, rejectedBy: string): Promise<void>;
  suspend(userId: string): Promise<void>;
  activate(userId: string): Promise<void>;
  deactivate(userId: string): Promise<void>;
  getAccountStatus(userId: string): Promise<AccountStatus>;
}
```

IPasswordService.ts

```
export interface IPasswordService {
  hash(password: string): Promise<string>;
  verify(password: string, hash: string): Promise<boolean>;
  update(userId: string, newPassword: string): Promise<void>;
  validateStrength(password: string): { valid: boolean; errors: string[] };
}
```

2. Single Responsibility (SRP)

Each service now has one clear responsibility:

- **UserReadService:** Handles all read operations (getById, getByEmail, list, etc.)
- **UserWriteService:** Handles all write operations (create, update, delete)
- **AuthenticationService:** Handles authentication logic (verifyCredentials, recordLogin, etc.)
- **AccountManagementService:** Handles account status changes (approve, reject, suspend, etc.)
- **PasswordService:** Handles password operations (hash, verify, validateStrength)

3. Dependency Inversion (DIP)

Created abstractions for external dependencies:

IUserRepository.ts

```
export interface IUserRepository {
  findById(id: string): Promise<User | null>;
  findByEmail(email: string): Promise<User | null>;
  findByFirebaseUid(firebaseUid: string): Promise<User | null>;
  findAll(filter?: UserFilter): Promise<User[]>;
  create(user: User, passwordHash: string): Promise<User>;
  update(user: User): Promise<void>;
  delete(id: string): Promise<void>;
  exists(email: string): Promise<boolean>;
  getPasswordHash(userId: string): Promise<string | null>;
  updatePasswordHash(userId: string, passwordHash: string): Promise<void>;
}
```

FirestoreUserRepository.ts

```
export class FirestoreUserRepository implements IUserRepository {  
  // Concrete implementation depending on abstraction (firestore module)  
  // Can be easily swapped with PostgreSQL, MySQL, etc.  
}
```

4. Open/Closed Principle (OCP)

Services are now open for extension but closed for modification:

- New user types can be added by extending **User** class and registering in container
- New authentication methods can be added by implementing **IAuthenticationService**
- New storage backends can be added by implementing **IUserRepository**
- No need to modify existing service code

5. Liskov Substitution Principle (LSP)

All implementations are substitutable for their interfaces:

```
// Can substitute any implementation  
const userReadService: IUserReadService = new UserReadService(userRepository);  
// Or for testing:  
const userReadService: IUserReadService = new MockUserReadService();
```

Dependency Injection Container

container.ts

```
export class Container {  
  private static instance: Container;  
  private services: Map<string, any> = new Map();  
  
  private constructor() {  
    this.initializeServices();  
  }  
  
  static getInstance(): Container {  
    if (!Container.instance) {  
      Container.instance = new Container();  
    }  
    return Container.instance;  
  }  
  
  private initializeServices(): void {  
    // Initialize repository (abstraction)
```

```

    const userRepository: IUserRepository = new FirestoreUserRepository();
    this.services.set('IUserRepository', userRepository);

    // Initialize services with dependencies injected
    const passwordService: IPasswordService = new
PasswordService(userRepository);
    const userReadService: IUserReadService = new
UserReadService(userRepository);
    const userWriteService: IUserWriteService = new
UserWriteService(userRepository, passwordService);
    const authenticationService: IAuthenticationService = new
AuthenticationService(userRepository, passwordService);
    const accountManagementService: IAccountManagementService = new
AccountManagementService(userRepository);

    this.services.set('IPasswordService', passwordService);
    this.services.set('IUserReadService', userReadService);
    this.services.set('IUserWriteService', userWriteService);
    this.services.set('IAuthenticationService', authenticationService);
    this.services.set('IAccountManagementService', accountManagementService);
}

get<T>(serviceName: string): T {
    const service = this.services.get(serviceName);
    if (!service) {
        throw new Error(`Service ${serviceName} not found in container`);
    }
    return service as T;
}
}

```

Facade Pattern

UserService.ts now acts as a facade, delegating to specialized services:

```

export class UserService {
    private container: Container;
    private userReadService: IUserReadService;
    private userWriteService: IUserWriteService;
    private authenticationService: IAuthenticationService;
    private accountManagementService: IAccountManagementService;

    constructor() {
        this.container = Container.getInstance();
        this.userReadService = this.container.getUserReadService();
        this.userWriteService = this.container.getUserWriteService();
        this.authenticationService = this.container.getAuthenticationService();
        this.accountManagementService =
this.container.getAccountManagementService();
    }
}

```

```
// Delegated methods maintain backward compatibility
async createUser(email: string, userType: UserType, profileData: any):
Promise<User> {
    return this.userWriteService.create(email, userType, profileData);
}

async verifyCredentials(email: string, password: string): Promise<User> {
    return this.authenticationService.verifyCredentials(email, password);
}

async approveUser(userId: string, approvedBy: string): Promise<void> {
    return this.accountManagementService.approve(userId, approvedBy);
}

// ... other delegated methods
}
```

Benefits Achieved

1. Single Responsibility Principle ☒

- Each service has one clear responsibility
- Changes are isolated to specific services
- Easier to understand and maintain

2. Open/Closed Principle ☒

- New user types can be added without modifying existing code
- New authentication methods can be plugged in
- New storage backends can be swapped

3. Liskov Substitution Principle ☒

- All implementations are substitutable for their interfaces
- Mock implementations can be used for testing
- Services work with abstractions, not concretions

4. Interface Segregation Principle ☒

- Small, focused interfaces
- Clients only depend on methods they use
- No "fat" interfaces forcing unnecessary dependencies

5. Dependency Inversion Principle ☒

- High-level modules depend on abstractions
- Low-level modules implement abstractions
- Dependencies are injected, not instantiated

- Easy to swap implementations (e.g., Firestore to PostgreSQL)

Testing Benefits

```
// Easy to mock for testing
describe('AuthenticationService', () => {
  it('should verify credentials', async () => {
    const mockUserRepository = {
      findByEmail: jest.fn().mockResolvedValue(mockUser),
      getPasswordHash: jest.fn().mockResolvedValue('hashedPassword')
    };
    const mockPasswordService = {
      verify: jest.fn().mockResolvedValue(true)
    };

    const authService = new AuthenticationService(
      mockUserRepository,
      mockPasswordService
    );

    const result = await authService.verifyCredentials('test@example.com',
    'password');
    expect(result).toBe(mockUser);
  });
});
```

Migration Strategy

Phase 1: Create Interfaces ☒

- Created all service interfaces
- Created repository interface
- Kept existing code working

Phase 2: Implement Services ☒

- Implemented specialized services
- Created DI container
- Updated one controller at a time

Phase 3: Refactor Domain ☒

- Made User class extensible
- Updated factory methods
- Maintained backward compatibility

Phase 4: Update Consumers ☒

- Updated routes to use new facade methods

- Fixed TypeScript compilation errors
- Maintained API compatibility

Files Created/Modified

New Files Created

1. `src/services/interfaces/IUserReadService.ts`
2. `src/services/interfaces/IUserWriteService.ts`
3. `src/services/interfaces/IAuthenticationService.ts`
4. `src/services/interfaces/IAccountManagementService.ts`
5. `src/services/interfaces/IPasswordService.ts`
6. `src/repositories/IUserRepository.ts`
7. `src/repositories/FirestoreUserRepository.ts`
8. `src/services/implementations/UserReadService.ts`
9. `src/services/implementations/UserWriteService.ts`
10. `src/services/implementations/AuthenticationService.ts`
11. `src/services/implementations/AccountManagementService.ts`
12. `src/services/implementations/PasswordService.ts`
13. `src/config/container.ts`

Files Modified

1. `src/services/UserService.ts` - Refactored to facade pattern
2. `src/routes/users.ts` - Updated to use new facade methods

Backward Compatibility

The refactoring maintains backward compatibility:

- Existing API endpoints continue to work
- `UserService` facade provides the same public interface
- Routes don't need to change (only internal implementation)
- Database schema remains unchanged

Future Improvements

1. **Add more user types:** Just extend `User` class and register in container
2. **Add caching:** Implement `ICacheService` and inject where needed
3. **Add event sourcing:** Implement `IEventStore` for audit trails
4. **Add microservices:** Split services into separate deployable units
5. **Add GraphQL:** Create GraphQL resolvers using the same service interfaces

Conclusion

The SOLID principles refactoring successfully transformed a monolithic `UserService` into a well-architected system with:

- Clear separation of concerns

- High cohesion within services
- Low coupling between services
- Easy testability
- Easy extensibility
- Maintainable codebase

The system now follows all five SOLID principles and is ready for future growth and changes.